OWASP Juice Shop
Web Application Penetration Test Report

Business Confidential

Date: August 12–16, 2026

Table of Contents

Table of Contents

Confidentiality Statement

This document is the exclusive property of **r.chandra sekhar** and contains confidential and proprietary information relating to a security assessment of the OWASP Juice Shop application. Reproduction or distribution of this document, in whole or in part, requires the express consent of **r.chandra sekhar**.

Disclaimer

A penetration test represents a snapshot in time. The findings and recommendations herein reflect the state of the application as observed during testing and do not account for any changes made afterward.

Time constraints do not permit exhaustive evaluation of every possible attack vector. Testing was prioritized toward the vulnerability classes most likely to be exploited by a real-world attacker, per the defined scope.

Contact Information

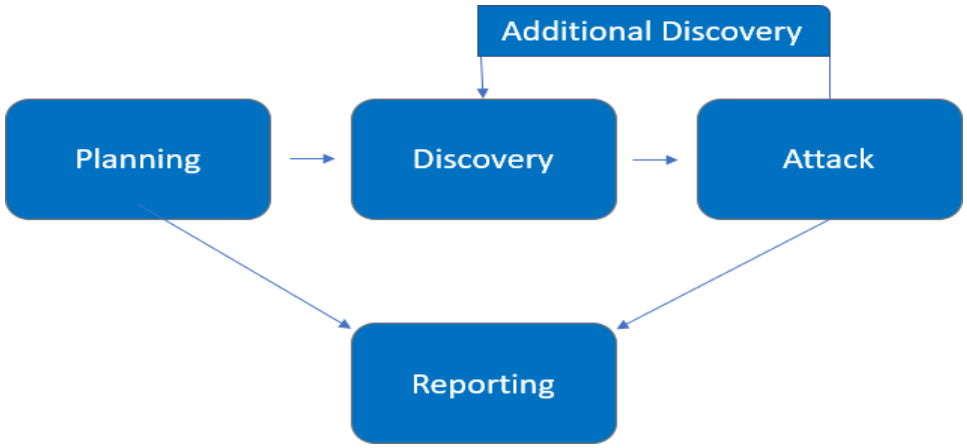
Name	Title	Contact Information
R.Chandra Sekhar	Penetration Tester (Report Author)	Email: chandur483@gmail.com

Assessment Overview

This engagement was a self-directed black-box penetration test of OWASP Juice Shop, an intentionally vulnerable web application, **conducted in an isolated local environment between August 12–16, 2026** . Testing covered both unauthenticated and authenticated attack surfaces and was performed manually, guided by the OWASP Web Security Testing Guide (WSTG) and PTES (Penetration Testing Execution Standard).

Phases of testing activity:

- **Planning** — Objectives, scope, and rules of engagement defined.
- **Discovery** — Enumeration of application functionality, endpoints, and technologies to map attack surface.
- **Attack** — Manual exploitation to confirm vulnerabilities; further discovery performed upon gaining new access (e.g., authenticated views).
- **Reporting** — Documentation of confirmed findings, tested-not-vulnerable results, and remediation guidance.



Assessment Components

Web Application Penetration Test

A web application penetration test emulates the actions of a real-world attacker with no privileged or insider knowledge of the target. Testing focuses on validating vulnerabilities across authentication and session handling, access control, input validation, and business logic — including injection flaws and broken object-level authorization (IDOR/BOLA). Testing extends beyond publicly visible functionality to authenticated areas of the application, in order to surface attack surface not exposed to an anonymous user.

Finding Severity Ratings

The following table defines levels of severity and corresponding CVSS score range that are used throughout the document to assess vulnerability and risk impact.

Severity	CVSS V3 Score Range	Definition
Critical	9.0-10.0	Exploitation is straightforward and usually results in system-level compromise. It is advised to form a plan of action and patch immediately.
High	7.0-8.9	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data or downtime. It is advised to form a plan of action and patch as soon as possible.
Moderate	4.0-6.9	Vulnerabilities exist but are not exploitable or require extra steps such as social engineering. It is advised to form a plan of action and patch after high-priority issues have been resolved.
Low	0.1-3.9	Vulnerabilities are non-exploitable but would reduce an organization's attack surface. It is advised to form a plan of action and patch during the next maintenance window.
Informational	N/A	No vulnerability exists. Additional information is provided regarding items noticed during testing, strong controls, and additional documentation.

Risk Factors

Risk is measured by two factors: Likelihood and Impact:

Likelihood

The probability of a vulnerability being discovered and exploited, based on attack complexity, tool availability, and required attacker skill level.

Impact

The effect of successful exploitation on the confidentiality, integrity, or availability of the application and its data.

Scope:

Assessment	Details
Web Application Penetration Test	**OWASP Juice Shop** (`http://localhost:3000`), **self-hosted via Docker** (`bkimminich/juice-shop`).

Scope Exclusions

The following were not performed:

- Denial of Service (DoS)
- Phishing/Social Engineering

All other testing not listed above was considered in scope.

Executive Summary

This report presents the results of a black-box penetration test conducted against OWASP Juice Shop, focused on evaluating the application's authentication, access control, and input-handling mechanisms. The assessment identified two critical-severity vulnerabilities — including an authentication bypass achieved through SQL injection — that would allow an unauthenticated attacker to gain full administrative access to the application.Five additional findings of moderate and informational severity were also identified ; immediate remediation of the critical-severity issues is strongly recommended.

Scoping and Time Limitations:

Testing was conducted between August 12–16, 2026 as a self-directed assessment , with no externally imposed time constraint. Denial-of-service and social-engineering techniques were excluded from testing, consistent with the Scope Exclusions defined above.

Testing Summary:

This assessment was conducted as a manual black-box penetration test against a locally hosted instance of OWASP Juice Shop, following the Vulnerability Analysis and Exploitation phases of the Penetration Testing Execution Standard (PTES), using Burp Suite as the primary interception and testing tool. Testing was carried out from two distinct threat-actor perspectives: an unauthenticated external attacker with no prior access, and an authenticated low-privileged user attempting to act outside their intended permissions.

Authentication was tested for resistance to injection-based login bypass, the strength of default and administrative credentials, and the application's resilience to automated password-guessing attempts.

Account registration was tested for information disclosure through response-based account enumeration, and for whether password policies stated in the interface were actually enforced by the server, rather than by the browser alone.

Session and object-level access control was tested from the authenticated low-privileged perspective, attempting to access and retrieve another user's private data by directly manipulating object identifiers rather than relying on the application's intended navigation.

Input handling was tested across multiple user-controllable fields — including search and user-submitted content — for injection and client-side script execution, exercising both the web front end and the underlying REST API directly rather than relying solely on the rendered interface.

Tester Notes and Recommendations

Across the confirmed findings, several patterns emerged that point to systemic gaps rather than isolated coding mistakes.

Inconsistent trust boundaries. The application enforces the same principle unevenly across the codebase — user-submitted content in the Product Review and Customer Feedback forms is correctly sanitized server-side, while the search functionality and the password-length policy trust client-side JavaScript alone. The controls needed clearly exist somewhere in the codebase; they simply aren't applied consistently to every user-controllable input.

Authorization checks scoped to authentication, not ownership. The IDOR finding shows the application correctly verifies that a user is *logged in*, but not that the logged-in user is *authorized* for the specific resource requested. This is a design-level gap, likely to recur on other object-identified endpoints beyond the one directly tested.

Single points of failure in authentication. A guessable administrative password was, on its own, sufficient for full account compromise — no secondary control such as rate limiting, account lockout, or multi-factor authentication existed to slow or stop the attempt. No single control failure should be enough to fully compromise an account.

Recommendations:

- Treat client-side validation strictly as a user-experience layer, never a security boundary — apply an equivalent server-side check for every client-side one currently in place.
- Extend the input-sanitization approach already used correctly on stored content (Product Review, Customer Feedback) consistently across all other user-controllable input, including URL parameters and search fields.
- Introduce a centralized, ownership-aware authorization check for object-identified endpoints, rather than relying on each endpoint to implement this correctly in isolation.
- Layer authentication defenses — rate limiting or account lockout combined with an enforced strong-password policy for administrative accounts — so no single control failure results in full compromise.

Key Strengths and Weaknesses

Strengths

- User-submitted content in the Product Review and Customer Feedback forms is correctly sanitized server-side, preventing persistent cross-site scripting through those channels.
- The login page returns an identical, generic error message regardless of whether the submitted email address is valid, correctly preventing account enumeration through that specific entry point.

Weaknesses

- Authentication is the most significant area of concern: the login form is vulnerable to a SQL injection bypass granting full administrative access without valid credentials, an issue compounded by a weak default administrative password and the absence of any rate limiting or lockout to slow repeated login attempts.
- Authorization checks do not consistently verify that a user owns the specific resource being requested, allowing one authenticated user to access another user's private data by directly manipulating an identifier.
- Several validation and sanitization controls exist only in the browser and are not re-enforced by the server, including password-length requirements and handling of user input in the search functionality.
- The account registration process discloses whether a given email address is already registered, aiding an attacker's reconnaissance ahead of a targeted attack.

Vulnerability Summary & Report Card

The following tables illustrate the vulnerabilities found by impact and recommended remediations:

2	0	3	0	2
Critical	High	Moderate	Low	Informational

Finding	Severity	Recommendation
WEB-01: SQL Injection — Authentication Bypass	Critical	Use parameterized queries/prepared statements for all database interactions; never concatenate user input into SQL.
WEB-02: Weak/Default Administrative Credentials	Critical	Enforce strong password requirements on administrative accounts and rotate default credentials immediately.
WEB-03: IDOR / Broken Object-Level Authorization — Basket Endpoint	Moderate	Implement ownership-based authorization checks on all object-identified endpoints.
WEB-04: DOM-based XSS via Search Functionality	Moderate	Sanitize all user input rendered to the DOM via Angular's built-in sanitizer; audit for unsafe HTML-binding overrides.
WEB-05: User Enumeration via Registration Endpoint	Moderate	Return identical, generic responses for registration attempts regardless of whether the email is already registered.
WEB-06: Missing Server-Side Password Length Validation	Informational	Enforce the stated password-length policy server-side, not client-side only.
WEB-07: Missing Account Lockout / Rate Limiting	Informational	Implement rate limiting or account lockout on the login endpoint.

Technical Findings

Web Penetration Test Findings

Finding WEB-03: Broken Object-Level Authorization — Basket Endpoint (Moderate)

Description:	Two low-privilege accounts were registered for testing: User A (hulk1@gmail.com, userId **25**, basket ID **6**) and User B (hulk2@gmail.com, userId **26**, basket ID **7**). Burp Suite's HTTP History was reviewed to identify requests referencing basket IDs during normal application use. While authenticated as User B, a request to `GET /rest/basket/6` — User A's basket — was captured and replayed via Burp Suite Repeater, without modifying User B's session token. The server returned the full contents of User A's basket (HTTP 200 OK).
Risk:	Likelihood: High — no special tooling required; any authenticated session can attempt sequential IDs. Impact: Medium — exposes another user's basket contents; write access was not confirmed in this test.
Affected Endpoint	GET /rest/basket/{id}
Tools Used:	Burp Suite Community Edition (Repeater)
References:	OWASP Top 10:2025 – A01 Broken Access Control , CWE-639 , WSTG-ATHZ-04

Evidence:

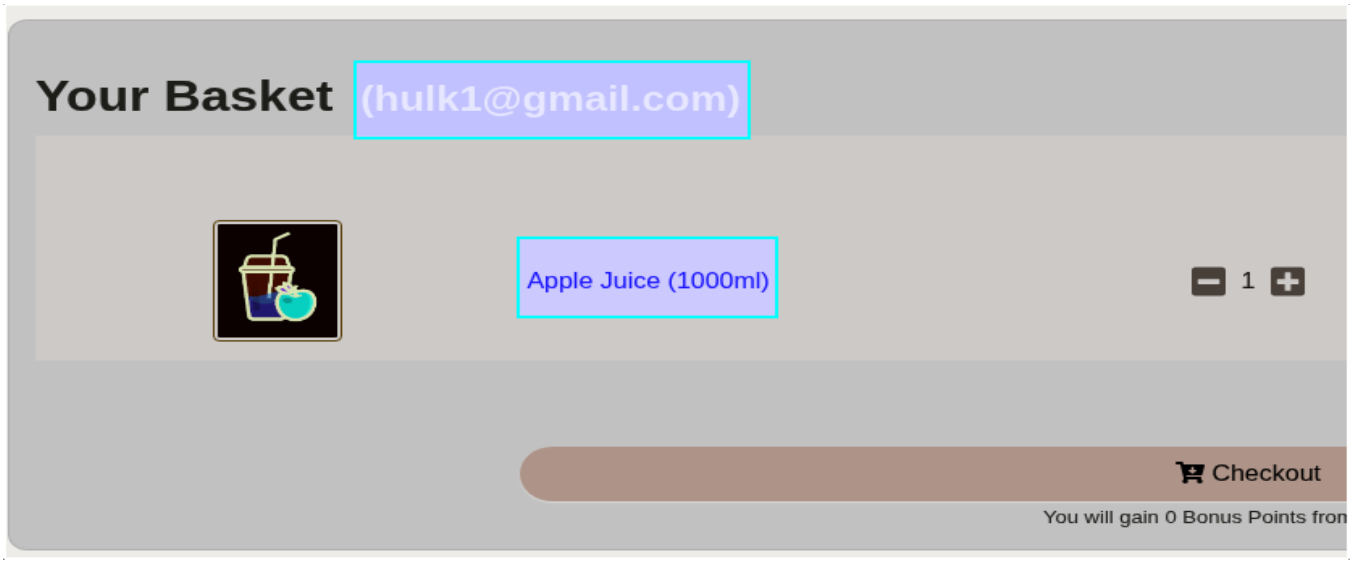


Figure 1 — hulk1's own basket.

Request

Pretty Raw Hex

```
1  GET /rest/basket/6 HTTP/1.1
2  Host: localhost:3000
3  sec-ch-ua-platform: "Linux"
4  Authorization: Bearer
   eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoyNSwidXNlcm5hb
   RmNDE2OGIwOWJlNGY1NGQxMmU4ZDNiZGNkYThhYmYiLCJyb2xlIjoiY3VzdG9tZXIiLCJk
   GVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiw
   MDI2LTA4LTE1IDEwOjUyOjA0LjM0NCArMDA6MDAiLCJ1cGRhdGVkQXQiOiIyMDI2LTA4LT
   iOjYsImlhdCI6MTc4Njc5MTE3N30.P7F_i5XjK05-OK08vG2rFHvE6ZZSo2jYawCs714YF
   713tOBkrpQ_OYJfFZxOcbR8MPpAAhQKbQTdSNLOcU9qRFOU_vNAcZ65El06v6dB2661wE
5  Accept-Language: en-GB,en;q=0.9
6  Accept: application/json, text/plain, */*
7  sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"
8  User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
9  sec-ch-ua-mobile: ?0
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Dest: empty
13 Referer: http://localhost:3000/
14 Accept-Encoding: gzip, deflate, br
15 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_statu
   q37B4Dk8naMjQ1R95mLbwdvXcYS55hqjtglcxz06PlEYZOoyVrgpzeXNxWv2; token=
   eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoyNSwidXNlcm5hb
```

Figure 2 — hulk1's own basket request.

Response

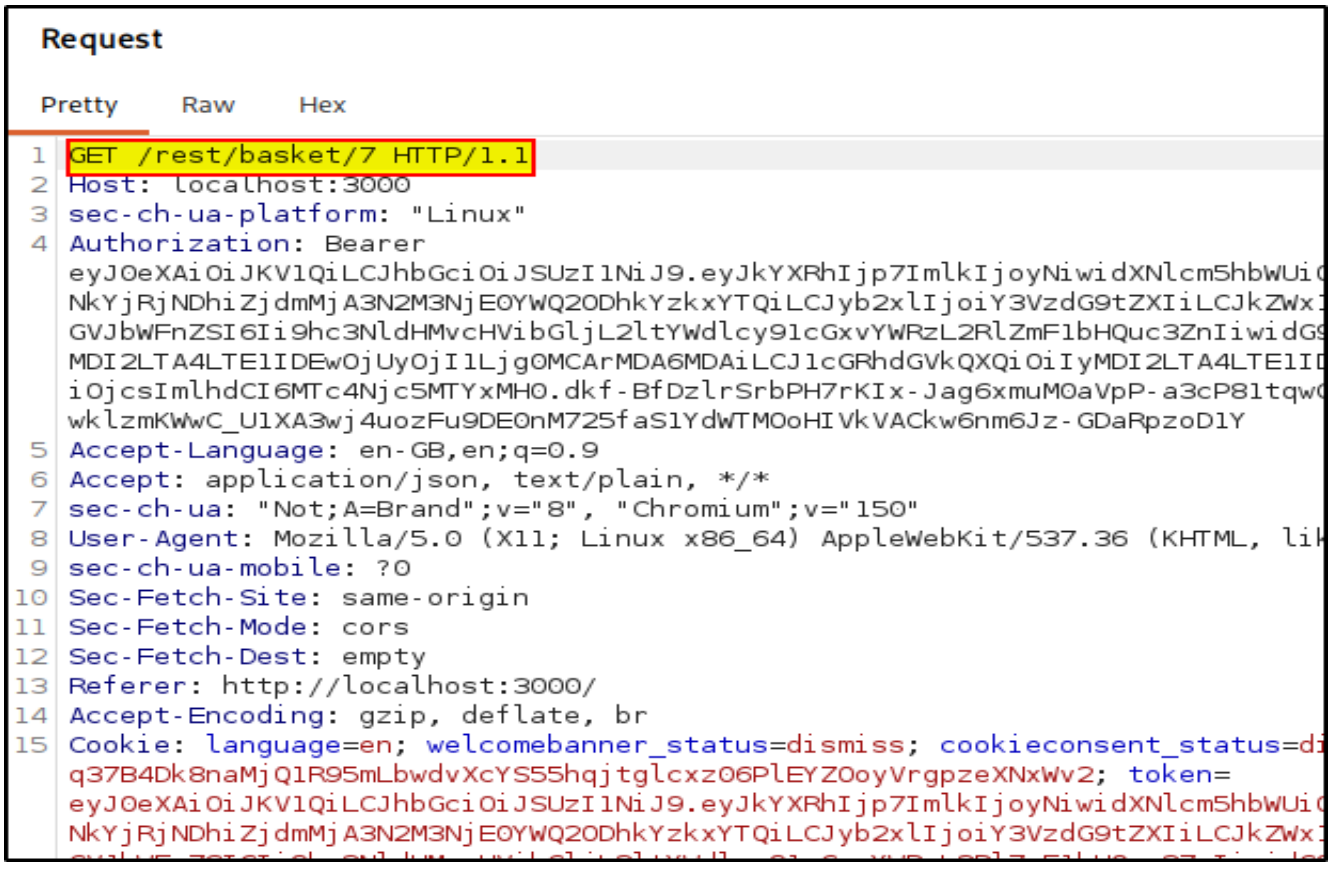
Pretty Raw Hex Render

```
1  HTTP/1.1 200 OK
2  Access-Control-Allow-Origin: *
3  X-Content-Type-Options: nosniff
4  X-Frame-Options: SAMEORIGIN
5  Feature-Policy: payment 'self'
6  X-Recruiting: /#/jobs
7  Content-Type: application/json; charset=utf-8
8  Content-Length: 523
9  ETag: W/"20b-q1WqVk3vc9tUvMTpbUHsw7sj7xk"
10 Vary: Accept-Encoding
11 Date: Sat, 15 Aug 2026 10:53:08 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
     "status":"success",
     "data":{
       "id":6,
       "coupon":null,
       "UserId":25,
       "createdAt":"2026-08-15T10:52:57.390Z",
       "updatedAt":"2026-08-15T10:52:57.390Z",
       "Products":[
         {
           "id":1,
           "name":"Apple Juice (1000ml)",
           "description":"The all-time classic.",
           "price":1.99,
           "deluxePrice":0.99,
           "image":"apple_juice.jpg",
           "createdAt":"2026-08-15T10:43:24.024Z",
           "updatedAt":"2026-08-15T10:43:24.024Z",
           "deletedAt":null,
           "BasketItem":{
             "ProductId":1,
             "BasketId":6,
             "id":9,
             "quantity":1,
             "createdAt":"2026-08-15T10:53:08.530Z",
             "updatedAt":"2026-08-15T10:53:08.530Z"
           }
         }
       ]
     }
   }
```

Figure 3 - hulk1's own basket response.

Figure 4 — hulk2's own basket.

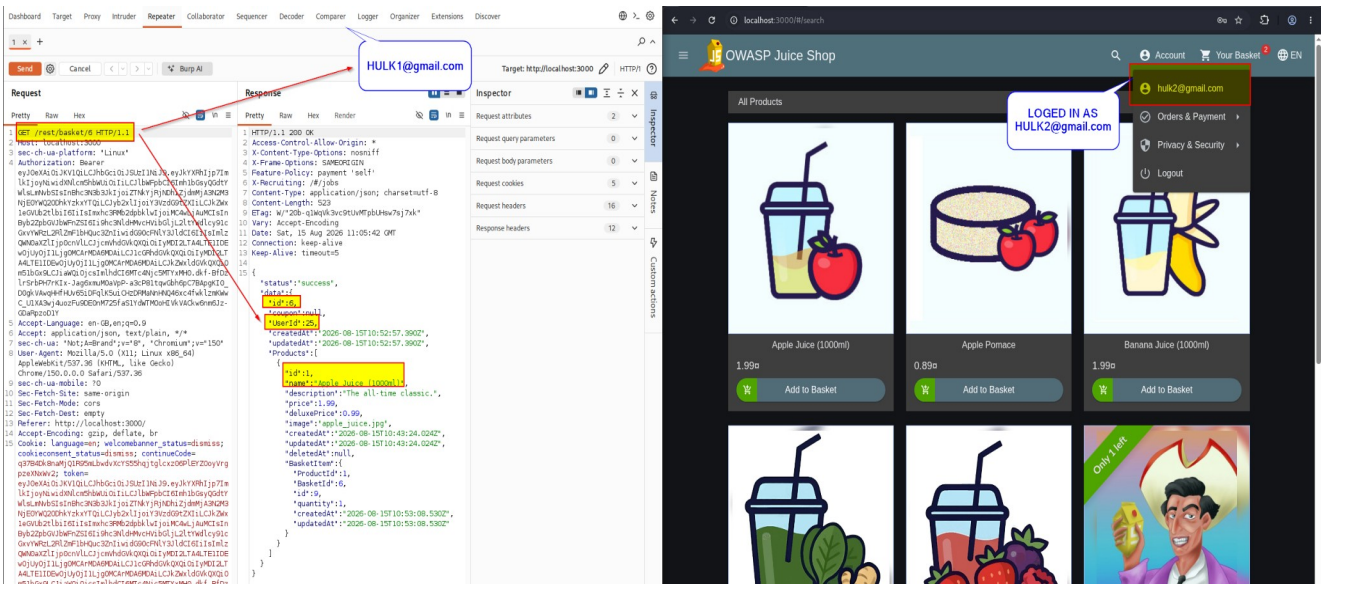
Figure 5 — hulk2's own basket request.

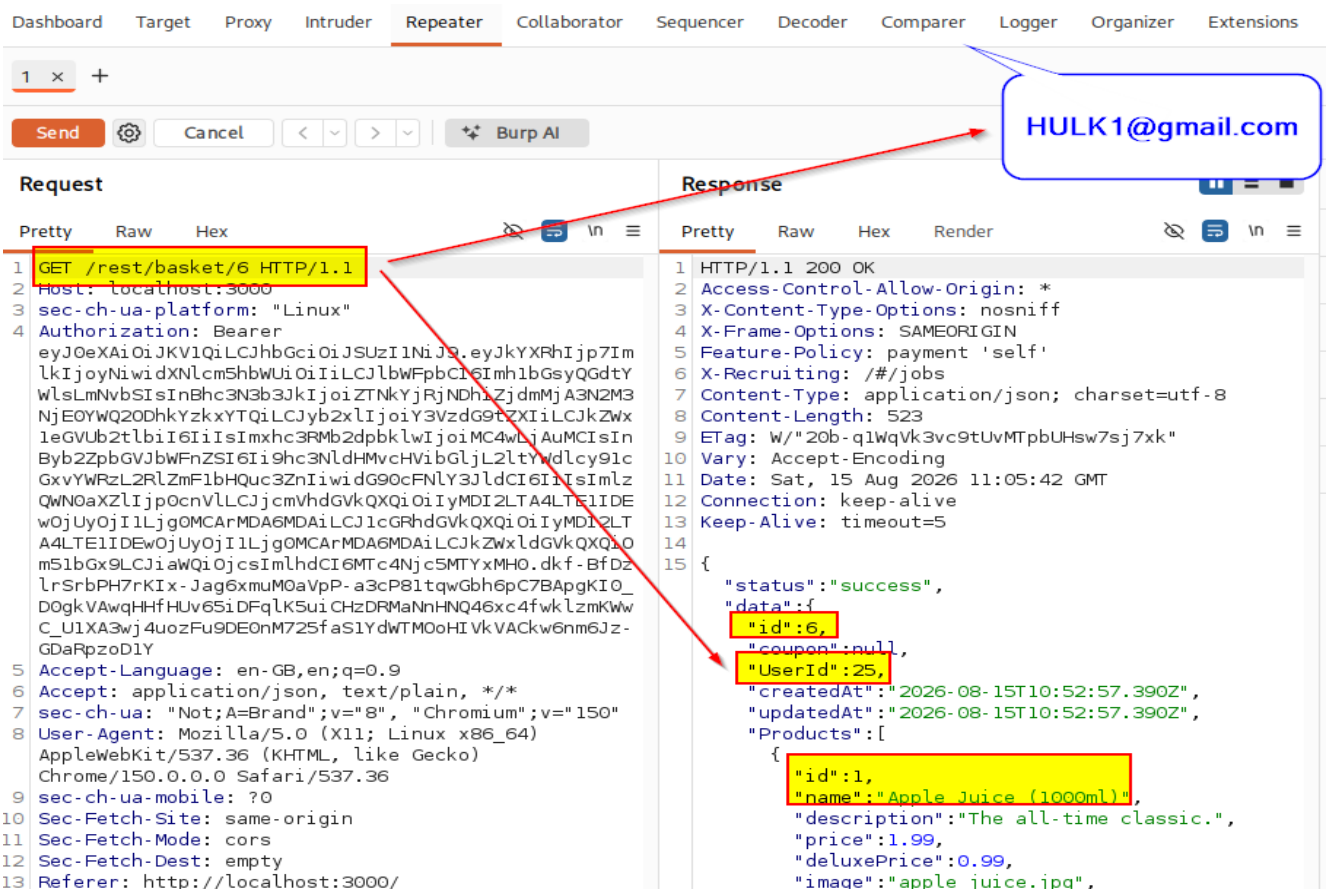


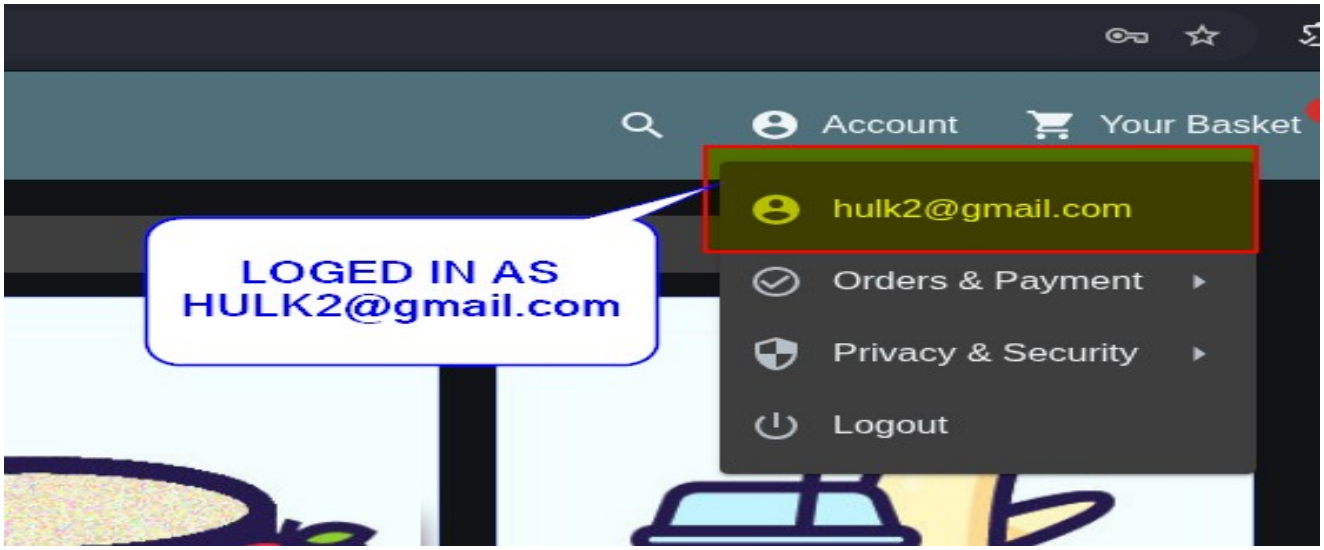
```
"status":"success",
"data":{
  "id":7,
  "coupon":null,
  "UserId":26,
  "createdAt":"2026-08-15T11:00:10.068Z",
  "updatedAt":"2026-08-15T11:00:10.068Z",
  "Products":[
    {
      "id":24,
      "name":"Apple Pomace",
      "description":
      "Finest pressings of apples. Allergy disclaim
      s</a> for recycling.",
      "price":0.89,
      "deluxePrice":0.89,
      "image":"apple_pressings.jpg",
      "createdAt":"2026-08-15T10:43:24.029Z",
      "updatedAt":"2026-08-15T10:43:24.029Z",
      "deletedAt":null,
      "BasketItem":{
        "ProductId":24,
        "BasketId":7,
        "id":10,
        "quantity":1,
        "createdAt":"2026-08-15T11:00:22.676Z",
        "updatedAt":"2026-08-15T11:00:22.676Z"
      }
    },
    {
      "id":6,
      "name":"Banana Juice (1000ml)",
      "description":"Monkeys love it the most.",
      "price":1.99,
      "deluxePrice":1.99,
      "image":"banana_juice.jpg",
      "createdAt":"2026-08-15T10:43:24.025Z",
      "updatedAt":"2026-08-15T10:43:24.025Z",
      "deletedAt":null,
      "BasketItem":{
        "ProductId":6,
        "BasketId":7,
        "id":11,
        "quantity":1,
        "createdAt":"2026-08-15T11:00:23.629Z",
        "updatedAt":"2026-08-15T11:00:23.629Z"
      }
    }
  ]
```


Figure 6 — hulk2's own basket response.

Figure 7 — the proof shot (hulk2's session pulling hulk1's basket via `/rest/basket/6`)







Remediation:

The /rest/basket/{id} endpoint must verify that the requesting user's session userId matches the userId associated with the requested basket, before returning any data. This check should apply to all basket operations — GET, PUT, DELETE — not only retrieval, since the underlying flaw is a missing ownership check at the endpoint level, not something specific to one HTTP method.

As a secondary hardening measure, replace sequential integer basket IDs with non-sequential, non-guessable identifiers (e.g. UUIDs). This reduces exposure to casual enumeration but does not substitute for the ownership check above — an attacker who obtains a valid ID through any other means would still succeed if that check isn't implemented.

Finding WEB-01: SQL Injection Leading to Authentication Bypass (Critical)

Description:	The login form's email field is used to construct a SQL query server-side without input sanitization or parameterization. Submitting ' OR 1=1-- caused the query's WHERE clause to evaluate true for any row, while the trailing -- discarded the remainder of the query — including the password check — resulting in successful authentication with no valid password. The same flaw was confirmed against a specific account using admin@juice-sh.op'--, which achieved the same bypass without requiring any password guess, authenticating directly as the admin user.
Risk:	**Likelihood: High** — No credentials, no account knowledge, and no special tooling required; a single copy-pasted payload against the login form — the most commonly targeted entry point on any web app — is sufficient. **Impact: Very High** — The bypass issues a valid, fully-privileged admin token, confirmed via direct access to the /administration panel and its user data — the same privileges as any legitimate admin account.
Affected Endpoint	POST /rest/user/login
Tools Used:	Burp + browser
References:	OWASP Top 10:2025 – A05:2025 Injection OWASP WSTG-ATHN-04: Testing for Bypassing Authentication Schema CWE-89: Improper Neutralization of Special Elements used in an SQL Command

Evidence:

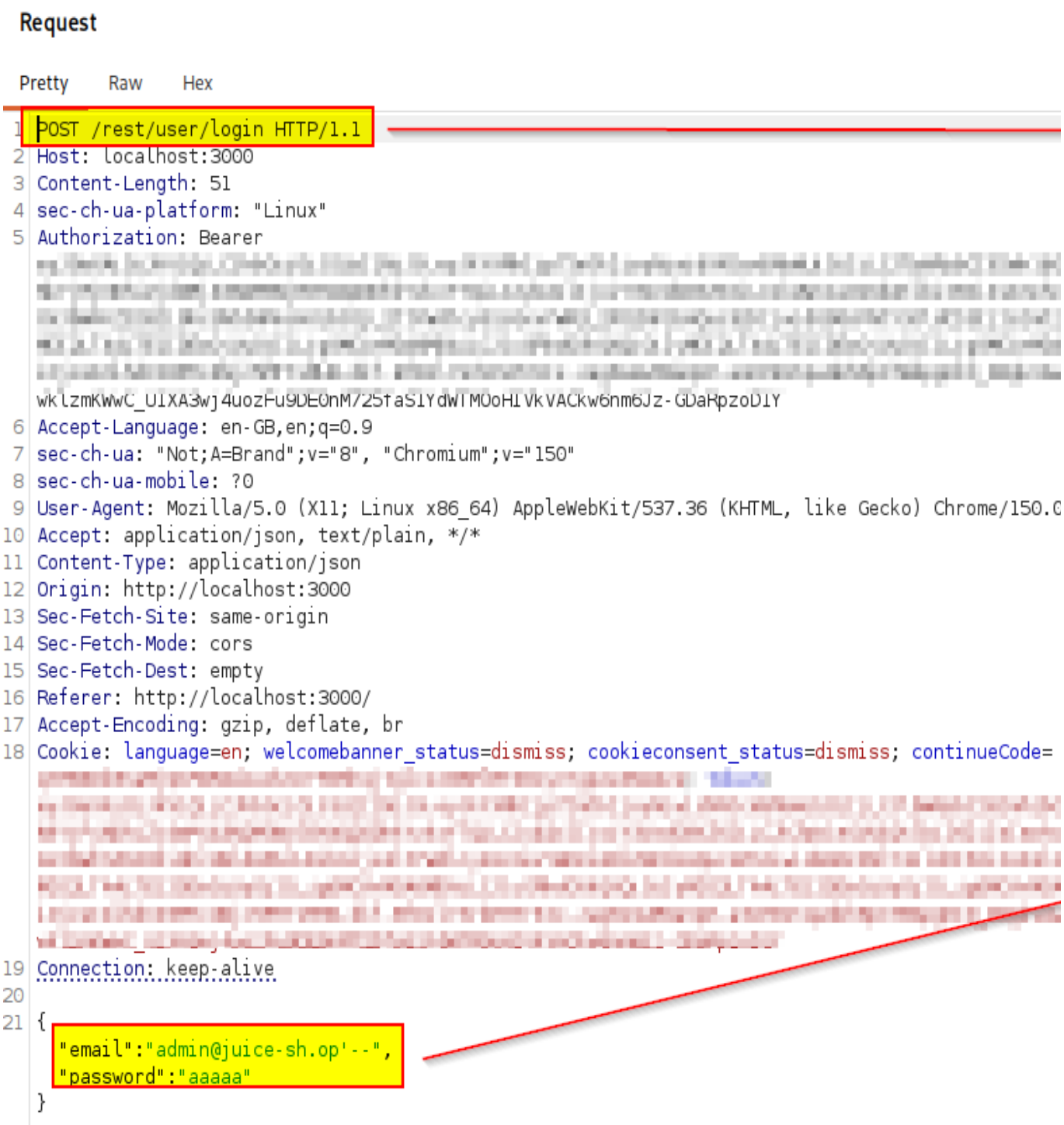


Figure 8 — `POST /rest/user/login` request containing the payload `admin@juice-sh.op'--` submitted in the email field.

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 200 OK
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 784
9 ETag: W/"310-3erNJJbLcERdK3hbXLSK5bidM2Y"
0 Vary: Accept-Encoding
1 Date: Sun, 16 Aug 2026 14:34:18 GMT
2 Connection: keep-alive
3 Keep-Alive: timeout=5
4
5 {
   "authentication":{
     "token":
       [redacted]
     "bid":1,
     "umail":"admin@juice-sh.op"
   }
}
```

Figure 9 — Server response: `HTTP/1.1 200 OK` with an issued authentication token and `"umail":"admin@juice-sh.op"`, confirming the session was authenticated as the admin account with no valid password supplied.

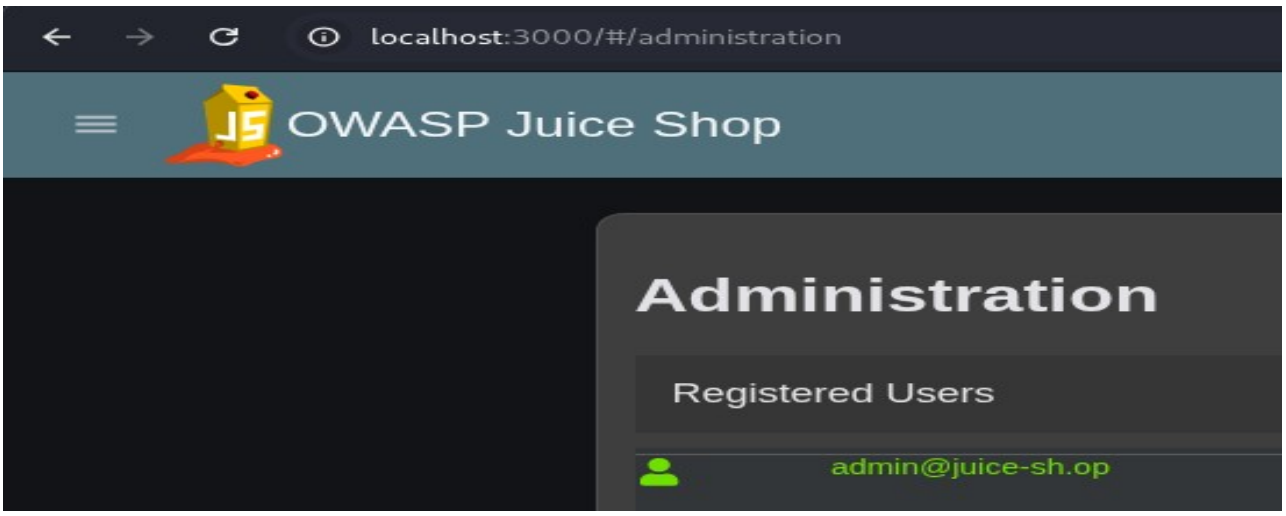


Figure 10 — *The `/administration` panel loading successfully post-bypass, confirming the bypass granted genuine admin-level access, not just a UI-level login state.*

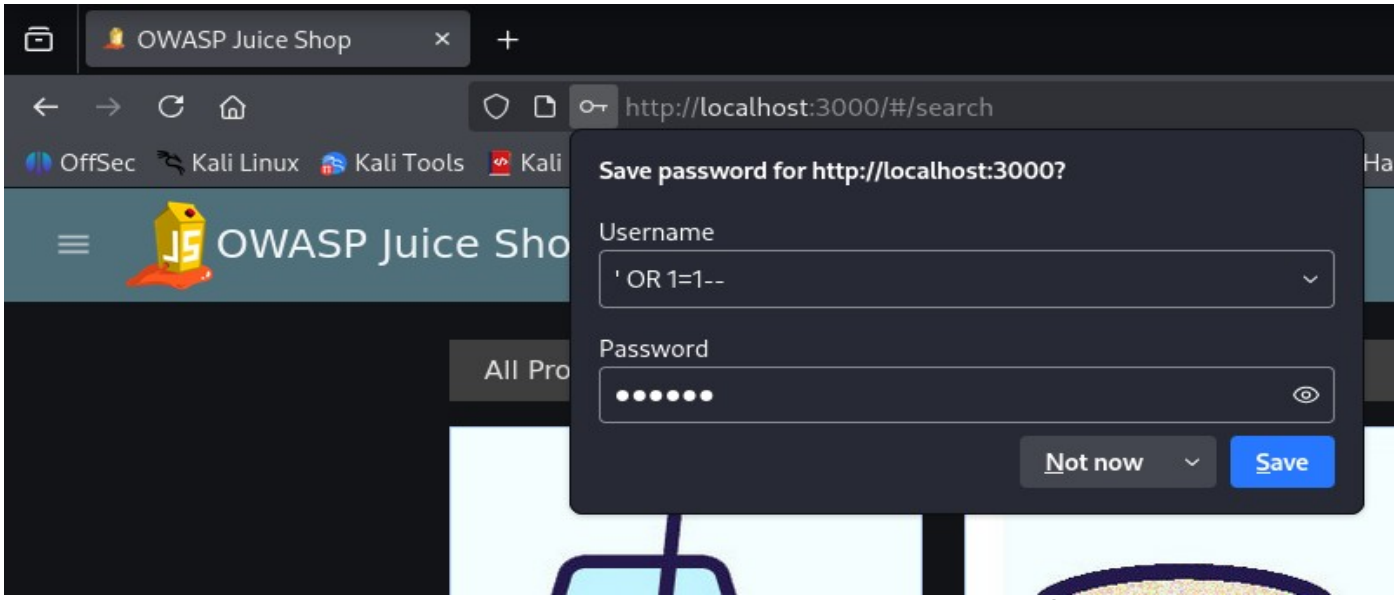


Figure 11 — *Browser password-save prompt after submitting the generic bypass payload `' OR 1=1--` in the login form.*

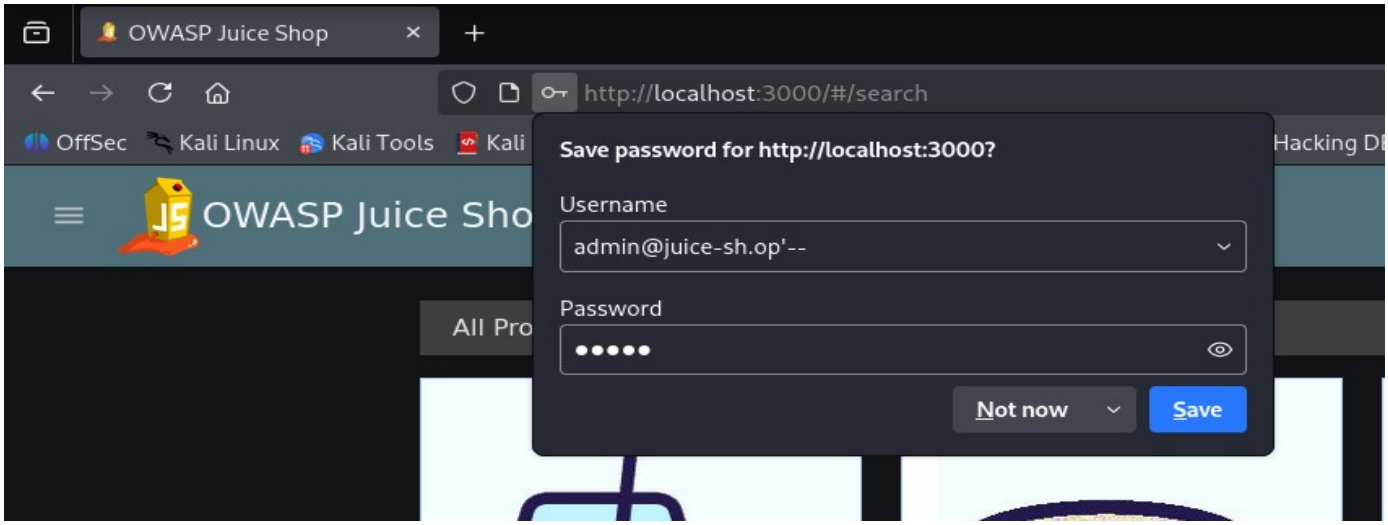


Figure 12 - *Browser password-save prompt after submitting the admin-specific bypass payload `admin@juice-sh.op'--` in the login form.*

Remediation:

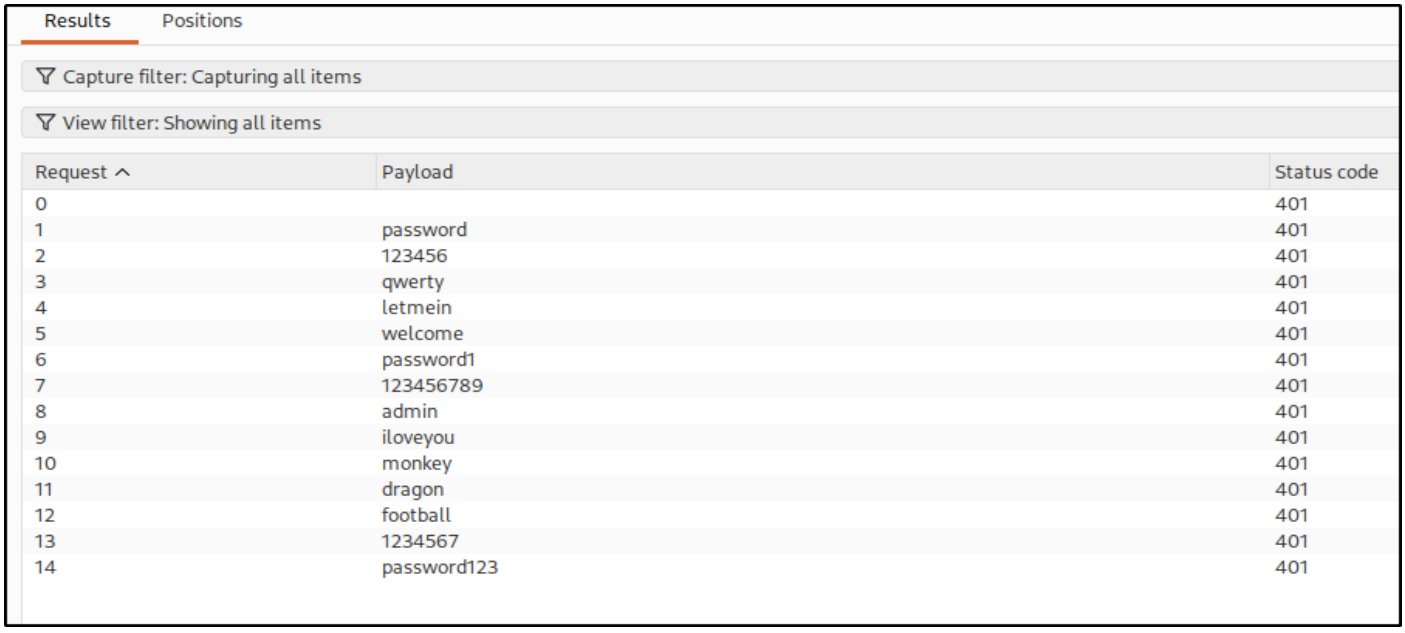
Primary fix: Use parameterized queries (prepared statements) instead of building SQL from raw string concatenation on /rest/user/login. This keeps user input as literal data — never part of the query's logic — no matter what's entered. Apply this fix everywhere else the app builds queries from user input, not just this one form.

Secondary hardening: Add input validation (e.g. proper email format checks) and run the database account on least- privilege permissions. Neither replaces parameterized queries — they only reduce damage if a similar flaw shows up elsewhere.

Finding WEB-07: Missing Account Lockout / Rate Limiting on Login Endpoint (Informational)

Description:	The login endpoint (POST /rest/user/login) does not enforce any account lockout, request throttling, or CAPTCHA challenge after repeated failed authentication attempts. Fourteen consecutive incorrect passwords were submitted against a known account (admin@juice-sh.op), and every attempt returned an unthrottled 401 Unauthorized, with no increasing delay, warning, or lockout observed. In the absence of this control, any account's password becomes guessable at whatever speed an attacker's tooling allows, with no resistance from the application itself
Risk:	**Likelihood: High** — attempting unlimited login guesses against this endpoint requires no privileges, no skill, and is fully automatable (Burp Intruder, Hydra, or a five-line script). This applies to every account on the application, not just the one tested. **Impact: Low, standing alone** — the missing control doesn't itself grant access; it removes the limit on how fast a weak password anywhere in the app could be found.
Affected Endpoint	POST /rest/user/login
Tools Used:	Burp **Intruder** (automated batch)
References:	OWASP Top 10:2025 – A07:2025 Authentication Failures — current category mapping CWE-307: Improper Restriction of Excessive Authentication Attempts — underlying weakness classification OWASP WSTG-ATHN-03: Testing for Weak Lock Out Mechanism — methodology reference for the test performed OWASP Authentication Cheat Sheet — supports Remediation

Evidence:



Request	Payload	Status code
0		401
1	password	401
2	123456	401
3	qwerty	401
4	letmein	401
5	welcome	401
6	password1	401
7	123456789	401
8	admin	401
9	iloveyou	401
10	monkey	401
11	dragon	401
12	football	401
13	1234567	401
14	password123	401

Figure 12 — Burp Intruder results for 14 consecutive invalid password attempts against `admin@juice-sh.op`***: every request returned*** `401 Unauthorized` ***with no CAPTCHA, block, or distinct lockout status observed across the sequence.***

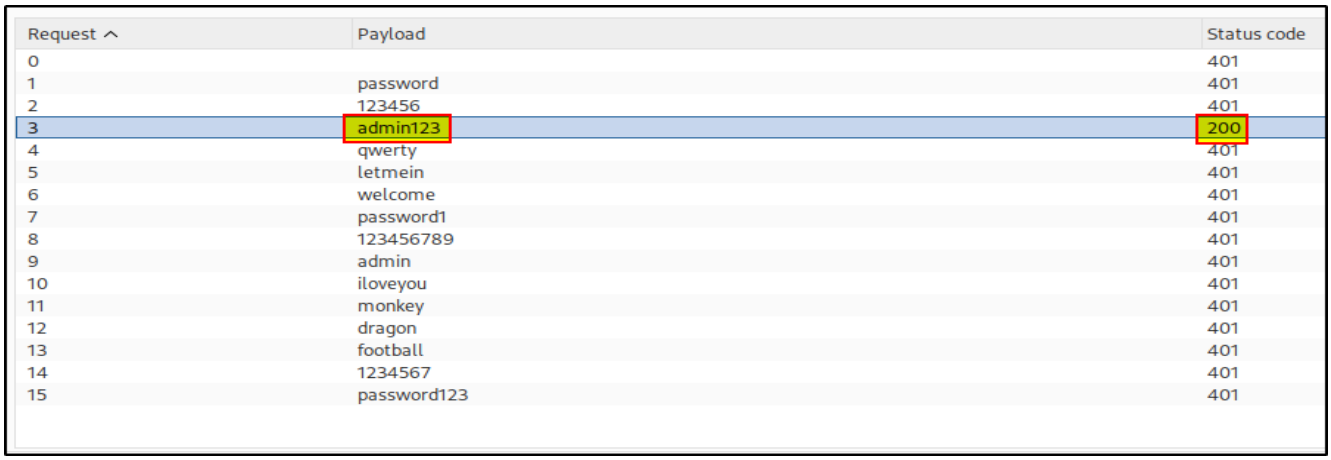
Remediation:

Primary fix: Implement server-side account lockout or progressive rate limiting on /rest/user/login — e.g., temporarily lock the account (or apply exponential backoff) after 5 consecutive failed attempts, tracked server-side per account and/or per source IP. This must be enforced server-side, not via client-side throttling, which can be bypassed by sending requests directly. Apply the same control to any other authentication-adjacent endpoint (e.g. password reset) where repeated guessing would otherwise go unchecked.
Secondary hardening: Add a CAPTCHA challenge after a low threshold of failed attempts, and enable monitoring/alerting on repeated failed logins to detect distributed or slow-rate brute-force attempts that a simple per-account lockout might miss. Neither replaces the lockout mechanism itself.

Finding WEB-02: Weak/Default Administrative Credentials (Critical)

Description:	The administrative account admin@juice-sh.op was found to use the password admin123 — a common, easily-guessable credential present in standard password wordlists (e.g. rockyou.txt). This was discovered by submitting a curated wordlist of common passwords against POST /rest/user/login via Burp Intruder, which returned a valid authenticated session for admin123, confirming successful compromise of the administrative account through credential guessing rather than any application logic flaw. This is made significantly more practical by the absence of rate limiting on the login endpoint (see WEB-07), which allows a wordlist attack to run uninterrupted.
Risk:	**Likelihood: High** — a curated wordlist attack against a known account is trivial to run, especially given the absence of rate limiting on the login endpoint (see WEB-07). **Impact: Very High** — the compromised account is the application's administrative account; this achieves the same impact as WEB-01 (full access to the /administration panel and user data), just via a different route (credential guessing instead of injection).
Affected Endpoint	POST /rest/user/login
Tools Used:	Burp **Intruder** (automated batch)
References:	OWASP Top 10:2025 – A07:2025 Authentication Failures — current category mapping , CWE-1391: Use of Weak Credentials — underlying weakness classification (corrected from CWE-521) , OWASP WSTG-ATHN-03: Testing for Weak Lock Out Mechanism — same methodology reference as the sibling finding OWASP Authentication Cheat Sheet — supports Remediation

Evidence:



Request ^	Payload	Status code
0		401
1	password	401
2	123456	401
3	admin123	200
4	qwerty	401
5	letmein	401
6	welcome	401
7	password1	401
8	123456789	401
9	admin	401
10	iloveyou	401
11	monkey	401
12	dragon	401
13	football	401
14	1234567	401
15	password123	401

Figure 13 — Burp Intruder results for a password-guessing attack against `admin@juice-sh.op`: the payload `admin123` returned `200 OK` while all other attempted passwords returned `401`, confirming the account's credential.

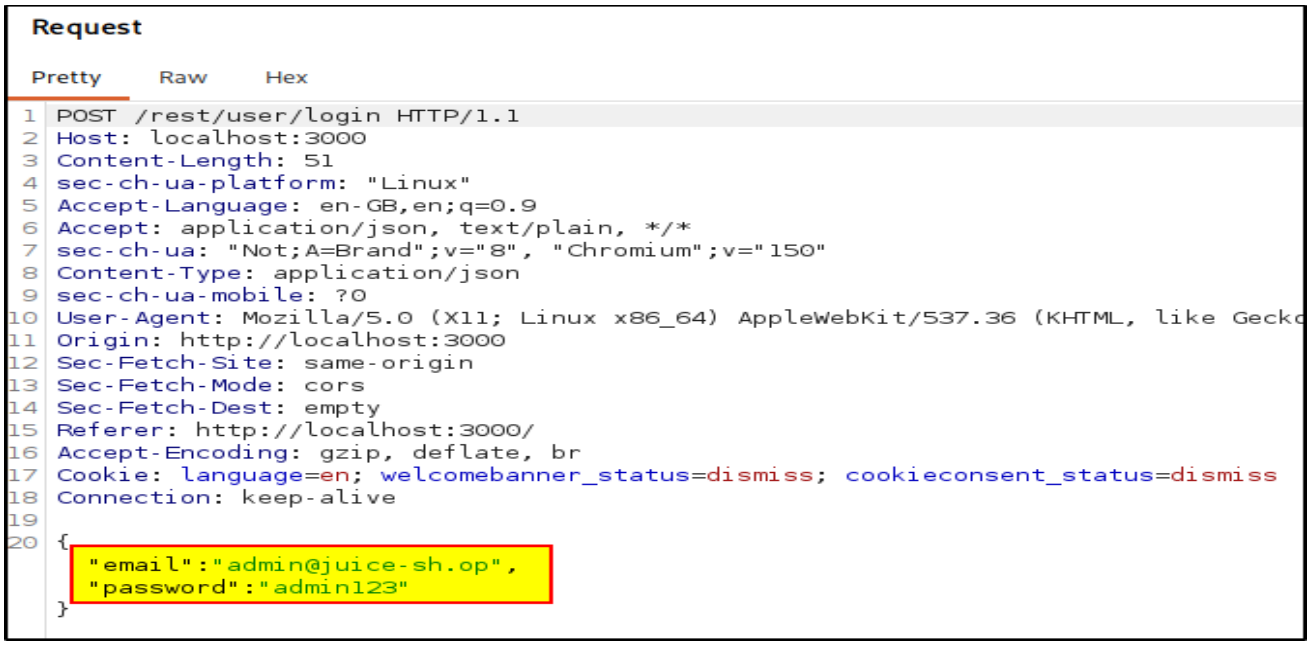


Figure 14 — `POST /rest/user/login` request submitting the confirmed credential (`admin@juice-sh.op` / `admin123`).

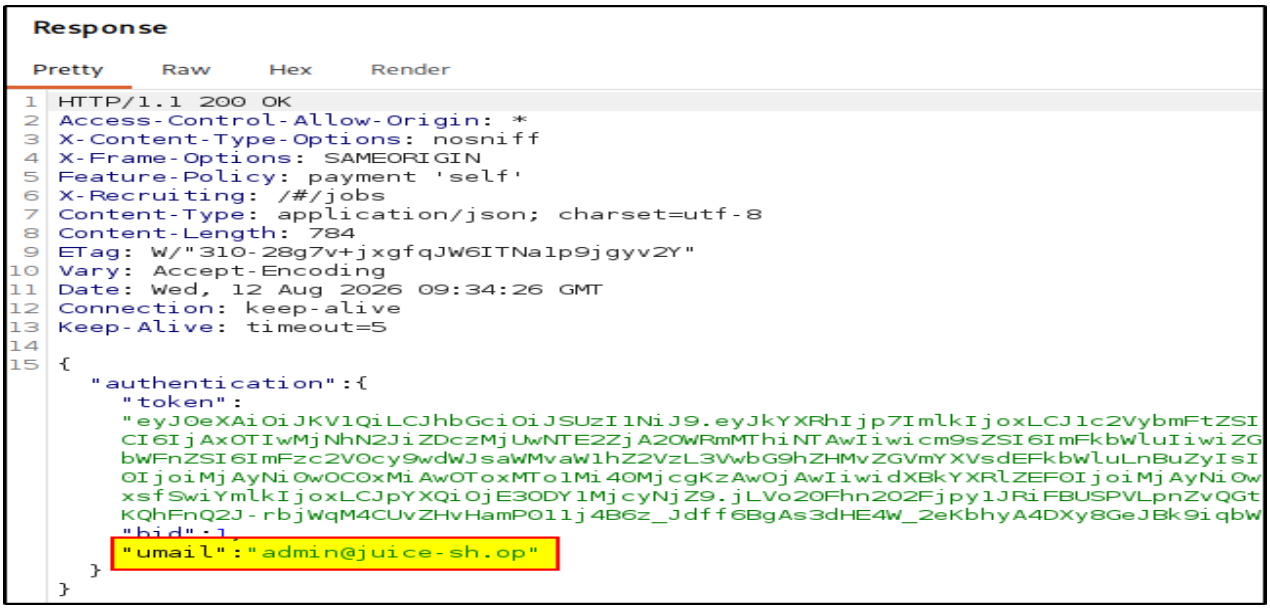


Figure 15- Response: *`HTTP/1.1 200 OK` with an issued authentication token and `"umail":"admin@juice-sh.op"`, confirming genuine administrative access was obtained.*

Remediation:

Primary fix: Rotate admin@juice-sh.op's password immediately. Enforce minimum password strength app-wide — at least blocking known-breached passwords (e.g. via rockyou.txt) — with strict enforcement on privileged accounts.
Secondary hardening: Add MFA on admin accounts. Paired with the rate-limiting fix (WEB-02), this closes both the "guessable" and "guessable at scale" halves of this problem.

Finding WEB-04: DOM-based Cross-Site Scripting via Search Functionality (Moderate)

Description:	Juice Shop's search functionality renders the URL's q query parameter directly into the page's DOM client-side, without sanitizing HTML/JavaScript content. A <script>alert(1)</script> payload did not execute — Angular's built-in sanitization blocks inline <script> tags inserted this way — but <img src=x onerror=alert(1)> executed successfully, confirming event-handler-based payloads bypass that protection. Because the payload is parsed and rendered entirely client-side, with no server round-trip involved, this is classified as DOM-based XSS rather than reflected XSS. **Note on manual confirmation:** Verified manually via direct browser interaction, not a Python script like WEB-01/02/03/07. Those findings could be confirmed by checking an HTTP response — achievable with requests alone. WEB-04 is DOM-based XSS: the payload only executes in the browser's JS engine after rendering, so no server response confirms it. Programmatic verification would need browser automation (e.g. Selenium), outside this toolkit's scope.
Risk:	**Likelihood: Medium** — requires an attacker to craft a malicious URL and get a victim to open it (e.g. via phishing or a shared link); unlike every other finding so far, this isn't zero-interaction — it requires a victim to click a malicious link, not just an attacker acting alone. **Impact: Medium** — proven code execution (alert(1)) confirms arbitrary JavaScript can run in a victim's session; this could realistically extend to session token theft or page content manipulation (e.g. phishing overlay), but only the proof-of-concept alert was demonstrated — actual data exfiltration wasn't tested, so that part is foreseeable risk, not proven impact.
Affected Endpoint	GET /rest/products/search?q=
Tools Used:	Browser only (manual URL/q= parameter manipulation)
References:	OWASP Top 10:2025 – A05:2025 Injection — same category as SQLi; CWE-79 confirmed as a member , CWE-79: Cross-site Scripting , OWASP WSTG-CLNT-01: Testing for DOM-based Cross-Site Scripting , OWASP DOM based XSS Prevention Cheat Sheet

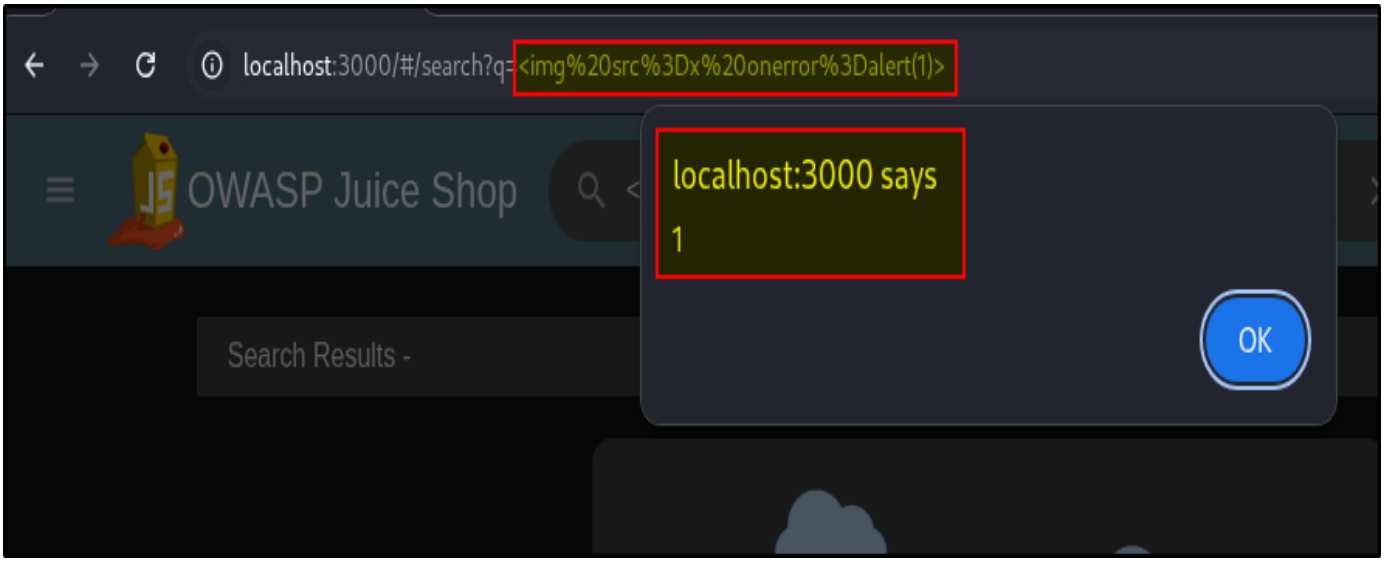


Figure 16- <img src=x onerror=alert(1)> ***submitted via the*** *q=* ***search parameter, triggering a JavaScript alert in the browser.***

Remediation:

Primary fix: Avoid rendering unsanitized user input into the DOM via innerHTML or equivalent bindings. Where dynamic HTML output is required, use Angular's DomSanitizer — audit for any use of bypassSecurityTrustHtml, since overriding sanitization is the most likely reason this payload executed.
Secondary hardening: Implement a Content-Security-Policy header restricting inline script/event- handler execution, and set the HttpOnly flag on session cookies to limit what a successful XSS payload could exfiltrate.

Finding WEB-06: Missing Server-Side Password Length Validation (Informational)

Description:	Juice Shop's registration form enforces a 5–40 character password policy via client-side JavaScript, rejecting passwords outside that range before submission. The registration request was captured in Burp Suite and replayed via Repeater with the password field replaced by 100 a characters, bypassing the browser's validation entirely. The server returned 201 Created and provisioned the account (id 26), confirming the upper bound of the policy is not enforced server-side. Whether the server independently enforces the 5-character minimum was not tested via direct request and is not claimed here.
Risk:	Likelihood: High — bypassing client-side validation requires only intercepting and editing one field in Burp; no privileges or special tooling needed. Impact: None, directly proven — accepting an over-length password doesn't itself expose or corrupt data. A foreseeable secondary risk exists if the underlying hashing algorithm's cost scales with input length (e.g. bcrypt), where accepting arbitrarily long passwords could be leveraged toward resource exhaustion at scale — not tested, not reflected in the CVSS score below.
Affected Endpoint :	POST /api/Users/
Tools Used:	Burp Suite (Repeater)
References:	OWASP Top 10:2025 – A06:2025 Insecure Design , CWE-602: Client-Side Enforcement of Server-Side Security , OWASP WSTG-ATHN-07: Testing for Weak Password Policy , OWASP Authentication Cheat Sheet

Evidence:


```json
{
 "email":"test4@gmail.com",
 "password":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
 "passwordRepeat":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
 "securityQuestion":{
   "id":1,
   "question":"Your eldest siblings middle name?",
   "createdAt":"2026-08-13T04:51:06.524Z",
   "updatedAt":"2026-08-13T04:51:06.524Z"
 },
```


Figure 17 –** `POST /api/Users/` **request with the password field replaced by 100** `a` **characters, and the server's** `201 Created` **response confirming account id 26 was provisioned.

Request

Pretty Raw Hex

```
POST /api/Users/ HTTP/1.1
Host: localhost:3000
Content-Length: 250
sec-ch-ua-platform: "Linux"
Authorization: Bearer
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoyOSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6Imh1bGsyQGdtYWlsLmNvbSIsInBhc3N3b3JkIjoiZT
NkYjRjNDhiZjdmMjA3N2M3NjE0YWQ2ODhkYzkxYTQiLCJyb2xlIjoiY3VzdG9tZXIiLCJkZWx1eGVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiMC4wLjAuMCIsInByb2Zpb
GVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiwidG90cFNlY3JldCI6IiIsImlzQWN0aXZlIjp0cnVlLCJjcmVhdGVkQXQiOiIy
MDI2LTA4LTEyIDExOjIzOjA4LjM3MiArMDA6MDAiLCJ1cGRhdGVkQXQiOiIyMDI2LTA4LTEyIDExOjIzOjA4LjM3MiArMDA6MDAiLCJkZWxldGVkQXQiOm51bGx9LCJpaWQ
iOjgsImlhdCI6MTc4NjUzNDE0MH0.cmg84Tu8cpto5NfNARzrURFSixIokUai02vDmC7iPfWXunrBTfX0jFv95KbANi8TmHNUlrNCqtEKcPD6vQgE73iN_Jr7wrNjrlvIF6
-i-8jgYdDRNdghW0g4GL4WG-LaZhdLB4ya7KYKl9iLWa7DHg9ySg3M5xwBhGkNkR93fgM
Accept-Language: en-GB,en;q=0.9
sec-ch-ua: "Not;A=Brand";v="8", "Chromium";v="150"
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
Accept: application/json, text/plain, */*
Content-Type: application/json
Origin: http://localhost:3000
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:3000/
Accept-Encoding: gzip, deflate, br
Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; continueCode=
yPMDQlKNBgW83akLEorY9zADPcQQh3RTjocLvAZOjp6124nwbqmx7ve5VRJX; token=
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp7ImlkIjoyOSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6Imh1bGsyQGdtYWlsLmNvbSIsInBhc3N3b3JkIjoiZT
NkYjRjNDhiZjdmMjA3N2M3NjE0YWQ2ODhkYzkxYTQiLCJyb2xlIjoiY3VzdG9tZXIiLCJkZWx1eGVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiMC4wLjAuMCIsInByb2Zpb
GVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHQuc3ZnIiwidG90cFNlY3JldCI6IiIsImlzQWN0aXZlIjp0cnVlLCJjcmVhdGVkQXQiOiIy
MDI2LTA4LTEyIDExOjIzOjA4LjM3MiArMDA6MDAiLCJ1cGRhdGVkQXQiOiIyMDI2LTA4LTEyIDExOjIzOjA4LjM3MiArMDA6MDAiLCJkZWxldGVkQXQiOm51bGx9LCJpaWQ
iOjgsImlhdCI6MTc4NjUzNDE0MH0.cmg84Tu8cpto5NfNARzrURFSixIokUai02vDmC7iPfWXunrBTfX0jFv95KbANi8TmHNUlrNCqtEKcPD6vQgE73iN_Jr7wrNjrlvIF6
-i-8jgYdDRNdghW0g4GL4WG-LaZhdLB4ya7KYKl9iLWa7DHg9ySg3M5xwBhGkNkR93fgM
Connection: keep-alive

{
  "email":"test4@gmail.com",
  "password":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
  "passwordRepeat":"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
  "securityQuestion":{
    "id":1,
    "question":"Your eldest siblings middle name?",
    "createdAt":"2026-08-13T04:51:06.524Z",
    "updatedAt":"2026-08-13T04:51:06.524Z"
  },
  "securityAnswer":"nancy "
}
```

Remediation :

Primary fix: Enforce the same 5–40 character policy server-side on /rest/user, independent of any client-side check. Since the underlying flaw is architectural — the server trusts client-side enforcement — apply the same check anywhere else in the app password length is currently validated only in the browser (e.g. password change/reset). Secondary hardening: Cap accepted request-body/field size at the framework level before input reaches the hashing routine, as general defense against oversized inputs — independent of, and not a substitute for, the policy fix above.

Finding WEB-05: User Enumeration via Registration Endpoint (Moderate)

Description:	Juice Shop's registration endpoint returns a distinguishing response depending on whether the submitted email address is already registered. Submitting a registration request for a known-existing account (admin@juice-sh.op) returned an "email must be unique" error, while submitting a fabricated, unregistered address (notarealuser12345@juice-sh.op) returned 201 Created. This discrepancy allows an unauthenticated party to confirm whether a specific email address has a registered account, without needing valid credentials. Notably, the application's *login* endpoint was separately tested and does not exhibit this weakness — it returns an identical generic error regardless of whether the submitted email exists (see Tested, Not Vulnerable).
Risk:	Likelihood: High — requires a single unauthenticated request per candidate email, fully automatable against a list of target addresses. Impact: Low — confirms account existence for a given email; does not expose account contents. Useful to an attacker chiefly as reconnaissance feeding into targeted phishing or credential-stuffing against confirmed real accounts.
Affected Endpoint:	POST /api/Users/
Tools Used:	Browser (registration form)
References:	OWASP Top 10:2025 – A07:2025 Authentication Failures , CWE-204: Observable Response Discrepancy , OWASP WSTG-IDNT-04: Testing for Account Enumeration and Guessable User Accounts , OWASP Authentication Cheat Sheet

Evidence:

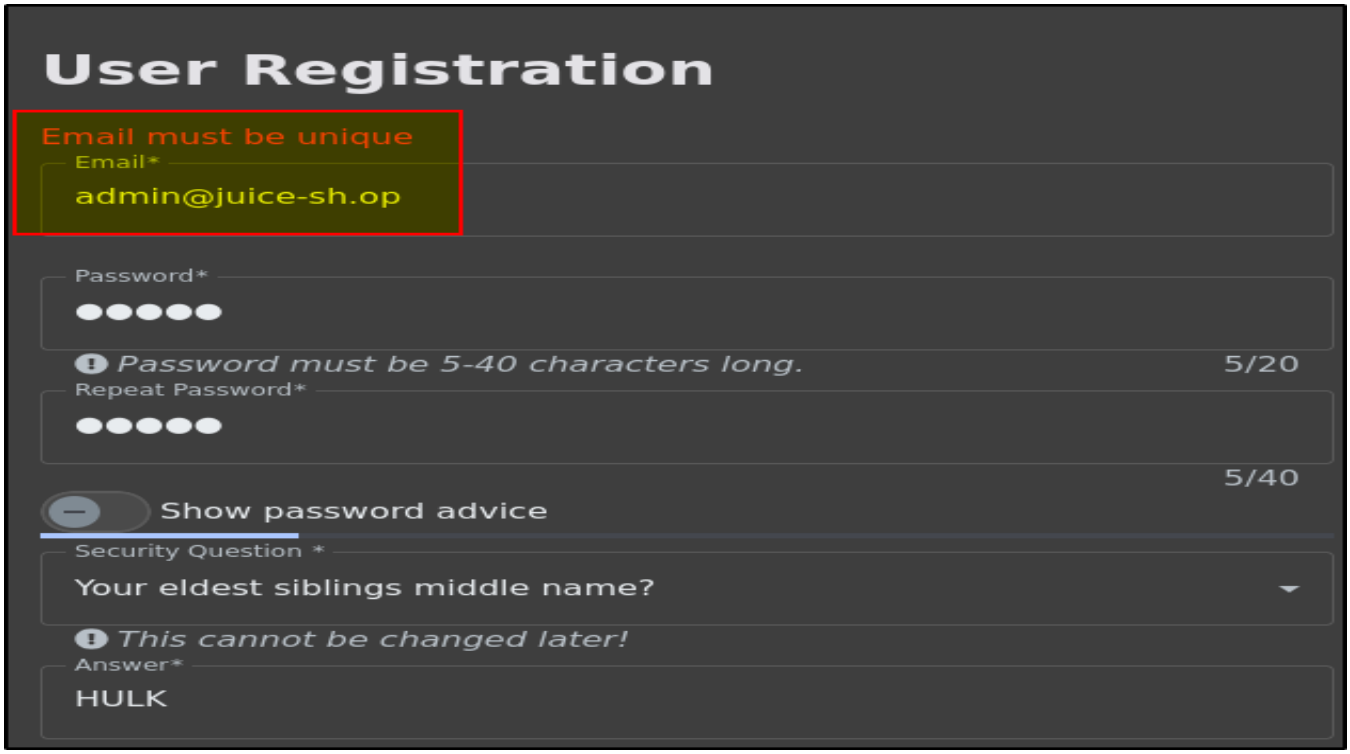


Figure 18 – Registration response for *admin@juice-sh.op* ***("email must be unique") alongside the*** *201 Created* ***response for the unregistered test address.***

Remediation:

Primary fix: Return an identical, generic response for both new and duplicate registration attempts — don't reveal uniqueness status directly. Juice Shop's own login page already does this correctly (generic error regardless of account validity); apply the same principle here.

Secondary hardening: Rate-limit and monitor repeated registration attempts from a single source, since automated enumeration depends on volume even against a fixed response.